TP12 — Grille de bataille et déplacements

Projet Java • Mini Tactical RPG inspiré de Fire Emblem • Niveau L1 • 10 exercices progressifs

Objectif général. Faire évoluer le moteur de bataille du TP11 vers un champ de bataille en deux dimensions : positions, limites, cases occupées et déplacements autorisés. À la fin du TP, une unité pourra se déplacer sur la grille sans traverser ses limites ni se superposer à une autre unité.

Point de départ. Conserver les classes Unite, Epeiste, Cavalier, Mage, Clerc, Partie et Main réalisées jusqu'au TP11. Les champs de position (ligne et colonne), ainsi que l'opération de changement de position déjà étudiée, peuvent être réutilisés ou adaptés si nécessaire.

Règles de travail. Aucun code solution n'est fourni. Chercher dans la documentation Java lorsque c'est nécessaire. Réaliser les exercices dans l'ordre, compiler et exécuter après chaque exercice, puis effectuer un commit et un push Git après validation. Ne pas utiliser de bibliothèque graphique dans ce TP.

Exercice 1 — Définir les dimensions du terrain

Travail demandé. Ajouter à Partie la notion de terrain rectangulaire avec un nombre de lignes et un nombre de colonnes. Les dimensions sont définies lors de la création d'une partie. Prévoir les opérations nécessaires pour les consulter.

Contrainte. Les dimensions appartiennent à la partie, et non aux unités. Une grille de 5 lignes et 7 colonnes contient des indices de lignes de 0 à 4 et de colonnes de 0 à 6.

Validation. Dans Main, créer une partie sur un terrain 5 × 7 et afficher correctement ses deux dimensions.

Exercice 2 — Donner une position initiale aux personnages

Travail demandé. Placer chaque personnage à une position de départ différente sur le terrain. Afficher, pour chaque unité, son nom, sa ligne et sa colonne. Réutiliser les informations de position disponibles dans Unite.

Contrainte. Ne pas créer un tableau d'indices qui remplace les positions des objets. Les coordonnées d'une unité doivent correspondre à sa position réelle.

Validation. À l'initialisation, les cinq personnages ont une position définie, distincte et située dans les limites de la grille.

Exercice 3 — Vérifier qu'une case existe

Travail demandé. Créer dans Partie une opération qui reçoit une ligne et une colonne et indique si cette case appartient au terrain.

Contrainte. Les indices négatifs sont interdits. Les indices égaux au nombre de lignes ou de colonnes sont également hors limites.

Validation. Sur un terrain 5 × 7, les cases (0,0) et (4,6) sont valides ; les cases (-1,2), (5,1) et (2,7) sont invalides.

Exercice 4 — Repérer une case occupée

Travail demandé. Créer dans Partie une opération qui vérifie si une case est déjà occupée par une unité. La recherche doit prendre en compte les alliés et les ennemis.

Contrainte. Comparer les positions des unités. Ne pas déduire l'occupation d'une case du nom d'un personnage ni d'un indice dans une liste. Pour ce TP, une unité à 0 PV occupe encore sa case.

Validation. Tester une case occupée par Chrom, une case occupée par Vaike et une case vide : les résultats doivent être cohérents.

Exercice 5 — Calculer la distance entre deux cases

Travail demandé. Créer une opération qui calcule la distance de Manhattan entre deux positions : nombre de déplacements horizontaux et verticaux nécessaires, sans diagonale. Chercher comment calculer la valeur absolue d'un entier en Java.

Contrainte. Ne pas utiliser de racine carrée : la distance diagonale euclidienne n'est pas celle demandée.

Validation. Entre (1,1) et (3,4), la distance vaut 5. Entre (2,2) et (2,2), elle vaut 0.

Exercice 6 — Déplacer une unité vers une case

Travail demandé. Créer dans Partie une opération permettant de demander le déplacement d'une unité vers une ligne et une colonne. Elle doit refuser une destination hors du terrain ou déjà occupée et indiquer si le déplacement a été effectué.

Contrainte. Réutiliser les vérifications des exercices 3 et 4. En cas de refus, les coordonnées de l'unité restent inchangées. Ne pas modifier directement les champs de position depuis Main.

Validation. Un déplacement vers une case vide du terrain réussit ; un déplacement vers une case occupée ou hors limites échoue sans changement de position.

Exercice 7 — Limiter la portée du déplacement

Travail demandé. Faire évoluer l'opération de déplacement : une unité ne peut se déplacer que vers une case située à une distance de Manhattan inférieure ou égale à 3 de sa position initiale.

Contrainte. La portée est commune à toutes les unités pour ce TP. Un déplacement de distance 0 n'est pas compté comme un déplacement réussi. Aucun cheminement autour des obstacles n'est demandé.

Validation. Depuis (1,1), atteindre (1,4) est autorisé si la case est libre ; atteindre (4,4) est refusé car la distance vaut 6.

Exercice 8 — Respecter l'état de l'unité

Travail demandé. Relier le déplacement à la gestion des activations de Partie. Un déplacement n'est autorisé que si l'unité peut agir. Après un déplacement réussi, l'unité passe à TERMINEE. Après un échec, elle conserve son état.

Contrainte. Réutiliser peutAgir() et la gestion existante des états. Ne pas dédupliquer leur logique. Une unité à 0 PV ne doit pas se déplacer.

Validation. Premier déplacement valide : accepté et unité TERMINEE. Deuxième déplacement pendant la même phase : refusé. Déplacement invalide initial : refusé et unité encore DISPONIBLE.

Exercice 9 — Afficher une grille textuelle

Travail demandé. Créer un affichage textuel du terrain, ligne par ligne. Une case vide est représentée par un point (.) ; une unité alliée par A et une unité ennemie par E. L'affichage doit tenir compte des positions actuelles.

Contrainte. Ne pas stocker une deuxième représentation indépendante des positions. Construire l'affichage à partir des unités présentes dans Partie. L'affichage ne doit pas modifier la partie.

Validation. Le nombre de lignes et de colonnes affichées correspond aux dimensions du terrain. Après un déplacement réussi, le symbole de l'unité apparaît à sa nouvelle position.

Exercice 10 — Construire une démonstration complète

Travail demandé. Nettoyer Main pour présenter un scénario court et lisible : création du terrain et des cinq unités, affichage de la grille initiale, tentative de déplacement hors limites, tentative vers une case occupée, déplacement valide, deuxième tentative de la même unité, affichage des états et de la grille finale.

Contrainte. Les décisions de déplacement doivent appartenir à Partie. Main organise les appels et présente les résultats. Ne pas ajouter d'interface graphique, d'intelligence artificielle ou de recherche de chemin.

Validation. Les refus ne modifient ni la position ni l'état. Un déplacement valide met l'unité à TERMINEE. La grille finale correspond exactement aux positions contenues dans les objets.

Critères de réussite du TP12. Le terrain est borné ; les unités ont des positions cohérentes ; les cases occupées sont prises en compte ; les déplacements respectent la portée et les états ; l'affichage reflète le modèle ; Main reste lisible.

Pour aller plus loin (hors évaluation). Réfléchir à la différence entre distance de Manhattan et chemin réellement praticable lorsqu'il existe des obstacles. Ne pas implémenter de recherche de chemin dans ce TP.